PROJECT REPORT

LSB Image Steganography

Developed in: C Programming Language

Architecture: Command-Line Interface (CLI)

Target Media: 24-bit Uncompressed BMP Images

1. INTRODUCTION

Steganography is the practice of concealing a file, message, image, or video within another file. The LSB (Least Significant Bit) Image Steganography project is a robust C-based utility designed to securely hide arbitrary secret files (such as `.txt` documents) inside standard 24-bit uncompressed BMP images. The primary objective of this system is to manipulate data at the bitwise level to embed payloads imperceptibly, demonstrating advanced knowledge of low-level memory management, file I/O operations, and binary data structures.

2. SYSTEM ARCHITECTURE & MODULES

The application is built using a highly modular architecture, separating the command-line interface, encoding logic, and decoding logic into distinct source files to ensure maintainability.

2.1. Main Controller (main.c)

The entry point validates command-line arguments and routes the execution flow to either the encoding or decoding module based on user flags (`-e` or `-d`). It utilizes customized structures like `EncodeInfo` and `DecodeInfo` to encapsulate file pointers and state variables, avoiding unsafe global variables.

2.2. Encoder Module (encode.c)

This module handles the embedding process. It first checks the image capacity by parsing the BMP header (reading dimensions at the 18-byte offset using `fseek`). It then sequentially embeds the magic string, file extension details, file size, and the raw payload into the image using bitwise substitution.

2.3. Decoder Module (decode.c)

This module reverses the process. It strips the LSBs from the stego-image, reconstructs the magic string to authenticate the file, and then dynamically decodes the payload size and content, writing the extracted data to a new output file.

3. ENCODING PROTOCOL & BITWISE LOGIC

To ensure data integrity during extraction, the system strictly follows a custom serialization protocol. Data is embedded in the following sequence:

- **BMP Header (54 Bytes):** Copied identically to preserve the image file structure.
- **Magic String (`#*`):** 16 bytes. Acts as a signature/password to authenticate the stego-image.
- **Secret File Extension Size:** 32 bytes. An integer denoting the length of the extension.
- **Secret File Extension:** Variable bytes. Defines the file type (e.g., `.txt`).
- **Payload Size:** 32 bytes. An integer defining the exact length of the secret data.
- **Payload Data:** Variable bytes. The actual secret message.

Core LSB Substitution Logic

The encoding process requires 8 bytes of image data to hide 1 byte of secret data. The algorithm isolates the target bit of the secret data, clears the LSB of the image byte, and merges them using the Bitwise OR operator.

```
Status encode_byte_to_lsb(char data, char *image_buffer) {
    for(int i=0; i<8; i++) {
        // 1. Extract the specific bit from the secret byte
        int get = ((data >> (7-i)) & 1);

        // 2. Clear the LSB of the image byte
        image_buffer[i] = (image_buffer[i] & 0xFE);

        // 3. Insert the secret bit
        image_buffer[i] = (image_buffer[i] | get);
    }
    return e_success;
}
```

4. TECHNICAL CONSTRAINTS & FILE I/O

Constraint	Implementation Details
Format Limitation	Strictly supports uncompressed BMP formats. Lossy formats like JPEG utilize Discrete Cosine Transform (DCT) compression, which averages out pixel data and destroys LSB alterations.
Capacity Checking	The system proactively calculates the maximum embeddable capacity using: `(Total_Image_Size - 54) / 8`. It aborts operations if the secret file exceeds this threshold, preventing file corruption.
State Management	All file pointers, string buffers, and integer capacities are safely bundled into the `EncodeInfo` struct, passed by reference to optimize stack memory usage.

5. LIMITATIONS AND FUTURE ENHANCEMENTS

While the LSB Steganography tool is fully functional, there are areas targeted for future optimization:

- **Variable-Length Array (VLA) Risk:** The current implementation defines the payload buffer dynamically on the stack (`char data[encInfo->size_secret_file];`). For massive files, this risks a stack overflow. Future iterations will utilize `malloc` for heap allocation or process the file in 1KB chunks via a streamed read/write pipeline.
- **Cryptographic Security:** The Magic String serves as an identifier but offers no cryptographic security. Implementing an AES encryption pass on the payload *before* LSB substitution would ensure absolute data confidentiality even if the stego-image is detected.
- **Payload Compression:** Integrating zlib to compress the secret file before embedding would significantly increase the effective storage capacity of the target image.

6. CONCLUSION

The LSB Image Steganography project successfully demonstrates advanced manipulation of raw binary data. By carefully controlling file pointers, utilizing complex bitwise arithmetic, and

structuring data into custom C structs, the project serves as a highly robust proof of concept for data obfuscation and low-level software engineering.